Data and Artificial Intelligence

Cyber Shujaa Program

Week 12 Assignment – Generative AI and Retrieval-Augmented Generation (RAG)

Student Name: Hope Njoki Nyambura
Student ID: cs-eh03-25100

Table of Contents

1. Introduction

This assignment demonstrates the application of Generative AI and Retrieval-Augmented Generation (RAG) to answer questions using context from a PDF document. The main objective was to implement a RAG pipeline that retrieves relevant text chunks and generates context-aware answers using a generative model (Flan-T5).

2. Data Description

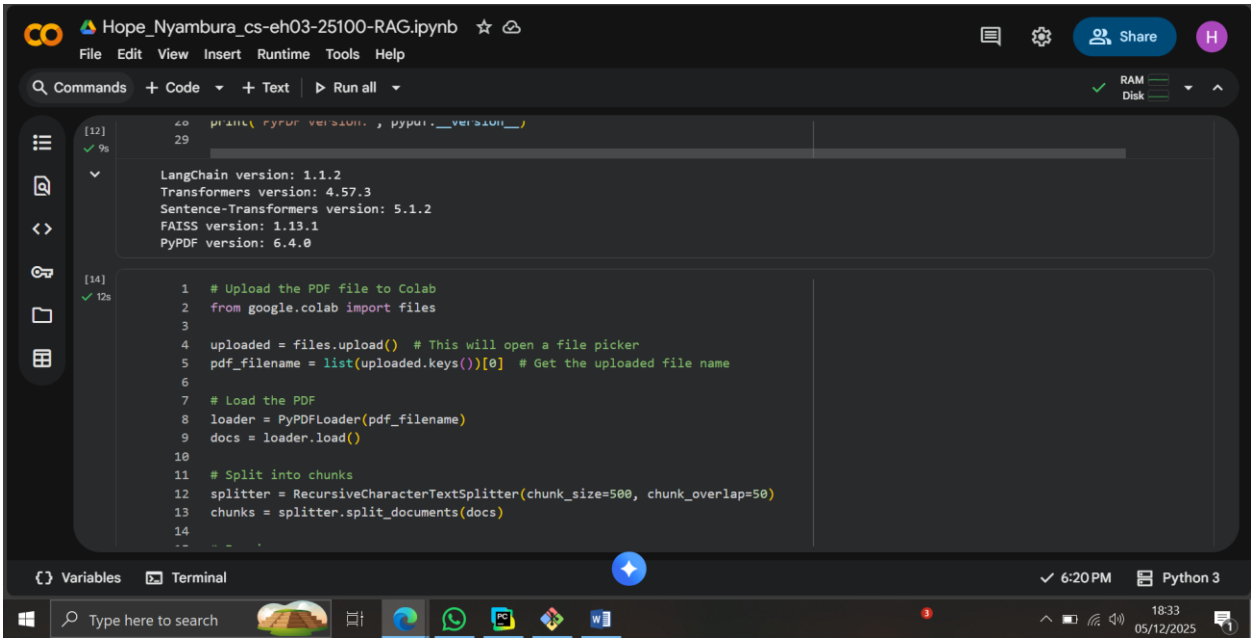
The input data consisted of a PDF document titled document.pdf. The document included content on Generative AI and RAG concepts. For processing:

- The PDF was split into smaller chunks of 500 characters with 50-character overlap to preserve context.
- A total of 6 chunks were generated.

3. Methods and Model Description

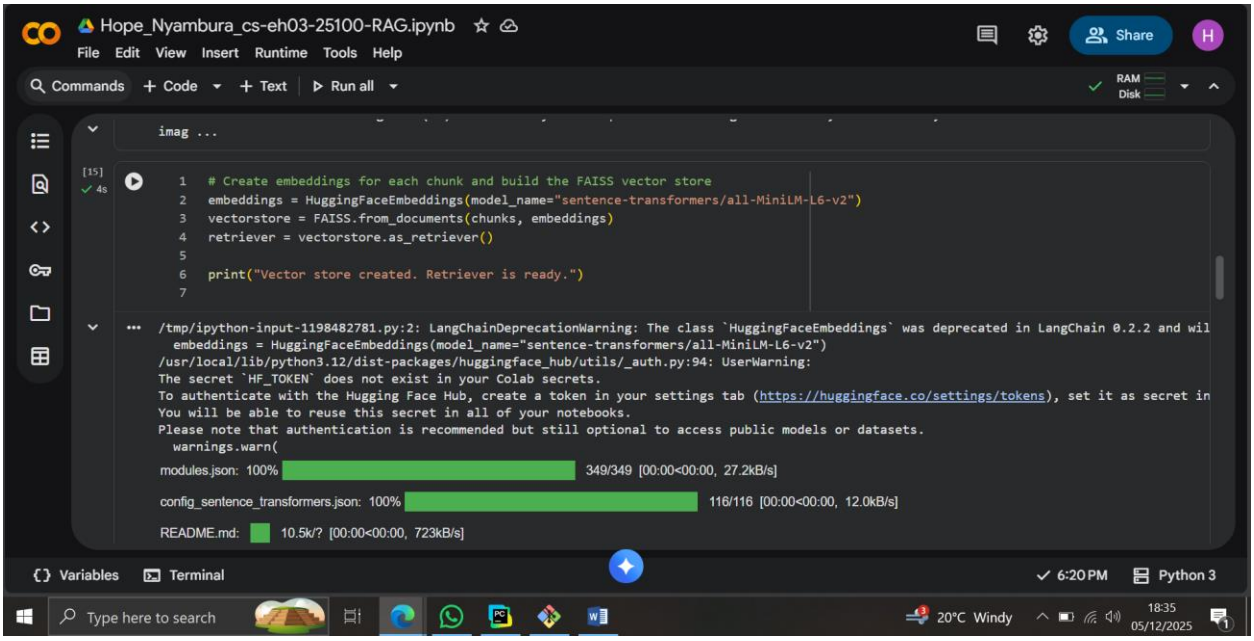
3.1 PDF Loading and Chunking

- Uploaded the PDF to Colab.
- Used PyPDFLoader to read the document.
- Split text into chunks using RecursiveCharacterTextSplitter.



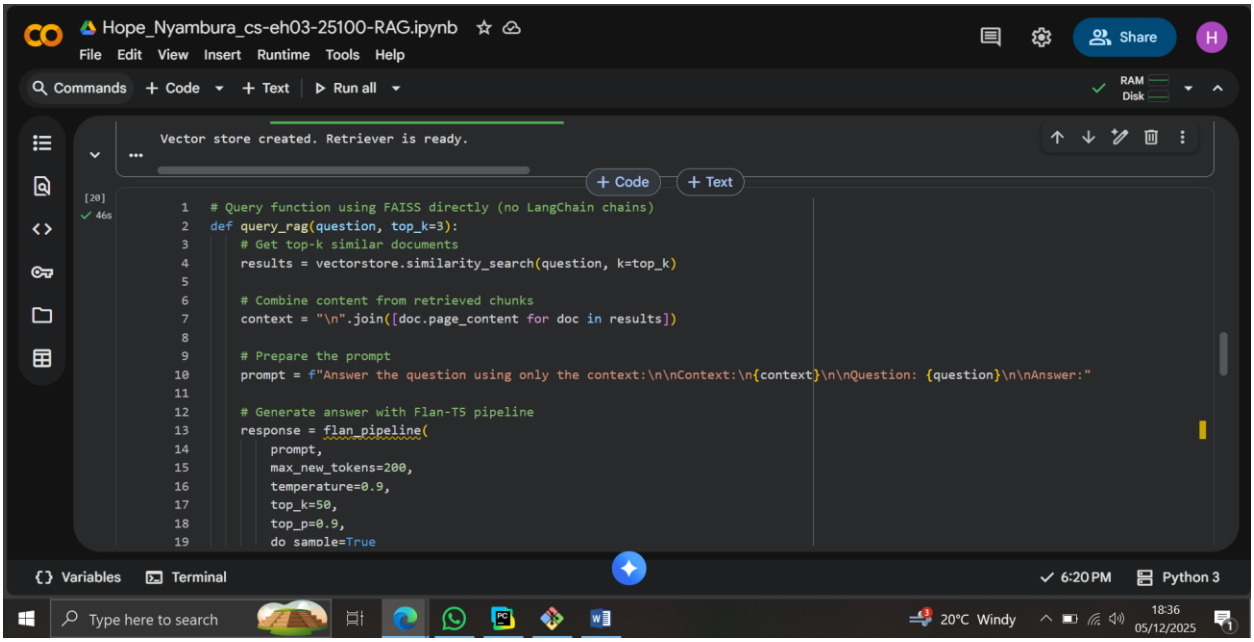
3.2 Embeddings and Vector Store

- Generated embeddings for each chunk using HuggingFace Sentence-Transformers (all-MiniLM-L6-v2).
- Built a FAISS vector store for efficient similarity search.
- Verified that the retriever was ready.



3.3 Generative Model (Flan-T5)

- Used Flan-T5 Large via HuggingFace pipeline for text generation.
- Configured for text2text generation with sampling parameters (temperature=0.9, top_k=50, top_p=0.9).



3.4 Retrieval-Augmented Generation (RAG)

- Implemented query_rag function to:
    1. Retrieve top-k relevant chunks from FAISS using similarity_search.
    2. Combine retrieved chunks as context.
    3. Generate answers with the Flan-T5 model.

4. Results

4.1 RAG Outputs

Example outputs from queries:

- **Query:** "Summarize the key points of this document in 200 words."
  Answer:

Retrieval-Augmented Generation (RAG) is an AI framework that combines two powerful components:
1. A retrieval system that fetches relevant documents.
2. A generative model that uses the retrieved information to produce accurate responses.
RAG helps AI overcome limitations such as hallucinations and outdated knowledge by grounding responses in real documents.

- **Query:** "What are the main steps in the RAG pipeline described in this document?"
  Answer:

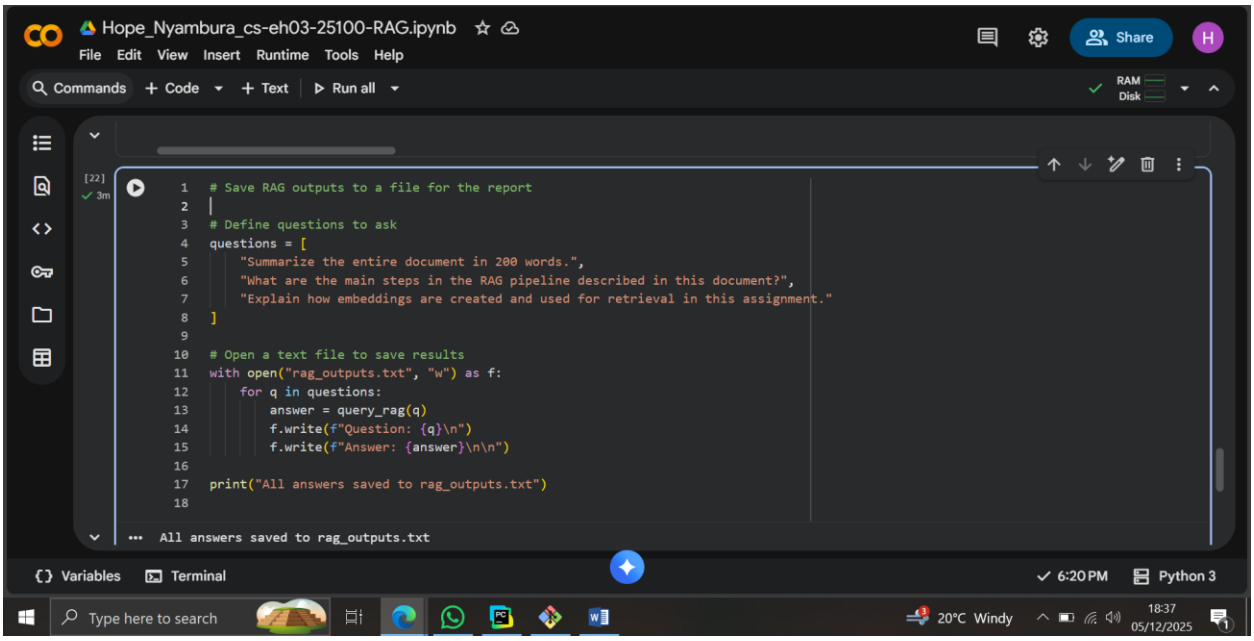
1. A user asks a question.
2. The system retrieves relevant text chunks from a knowledge base.
3. The retrieved information is combined with the question.
4. The generative model produces an answer based on the retrieved context.

- **Query:** "Explain how embeddings are created and used for retrieval in this assignment."
  Answer:

Embeddings were generated for each document chunk using Sentence-Transformers (all-MiniLM-L6-v2).
These embeddings represent the semantic meaning of each chunk in vector form.
The FAISS vector store uses these embeddings to retrieve the most relevant chunks for a given question.

4.2 Saved Outputs

- All outputs were saved in rag_outputs.txt for submission and report purposes.



5. Key Concepts

1. **Generative AI** – Systems that create new content, e.g., text or images.
2. **RAG** – Combines retrieval and generative models for context-aware answers.
3. **FAISS** – Efficient similarity search library for vector embeddings.
4. **Sentence Embeddings** – Vector representations of text chunks capturing semantic meaning.
5. **Flan-T5** – Generative transformer model used for producing answers from context.

6. Conclusion

- Successfully implemented a RAG pipeline in Python using LangChain, FAISS, and Flan-T5.
- Demonstrated retrieval of relevant document chunks and generation of context-aware answers.
- Learned how embeddings and vector stores improve generative model accuracy.
- Gained hands-on experience with PDF processing, vector databases, and generative AI pipelines.

7. Google Colab Link

[https://colab.research.google.com/drive/19ThxzMG2kw3nA27oa16u21g4BgBe9Q3j?usp=sharing]